

packet transmission delay: $L \text{ (bits)} / R \text{ (transmission rate) (bps)}$
 propagation delay = length of physical link / propagation speed
 nodal proc delay = check for errors & determine output, $< \text{msec}$
 queuing delay = time waiting for transmission, depends on congestion

throughput: the rate (bits/time) at which bits are transferred between sender & receiver

goodput: # of useful bits / time; excludes protocol overhead bits and retransmitted data packets

internet protocol layers:

1. **application**: supports network applications (HTTP, SMTP)
2. **transport**: processes data transport (TCP, UDP)
3. **network**: routing data from source to destination (IP)
4. **link**: data transfer between neighboring networks (WiFi)
5. **physical**: bits "on a wire"

transmission control protocol (TCP): connection-oriented, handshake required between client and server; flow control (sender can't overwhelm receiver); congestion control (throttles sender when network is overloaded); slower due to error checking and congestion control; use when reliability $>$ speed; use cases: web browsing (HTTP), remote access (SSH), database connections

user datagram protocol (UDP): connectionless; less reliable since no guarantee of delivery or error checking; faster (less overhead); use when speed $>$ reliability; use cases: live streaming, online gaming, DNS lookups

nonpersistent HTTP: only one object per TCP connection

persistent HTTP: multiple objects in a TCP connection

user agent: composes, edits, reads messages,

outgoing + incoming messages stored in server

mail server: contains incoming messages and

queue of outgoing messages

SMTP protocol: between mail servers to send

emails, uses TCP and persistent HTTP (reliable)

N = number of file copies

E/u_s = time to send one copy

$d_{\min}$ = min download time

$F/d_{\min}$ = min download time

client/server file distribution time:

$\max(NF/u_s, F/d_{\min})$

peer-to-peer distribution time:

$\max(E/u_s, F/d_{\min}, NF/(u_s \cdot \lambda))$

circuit switching: one dedicated line for each sender and receiver; no

packet loss; reliable bandwidth for audio/video streaming

packet switching: data is broken into packets, sent independently

across network, and reassembled at destination

store & forward: type of packet switching where intermediate nodes

receive packet, store it temporarily, check for errors, then forward to next node; total delay is $2L/R$

client/server architecture: server always on host; permanent IP address; data centers for scaling; clients connect with server not each other

peer-to-peer: no always on server; peers request from and provide to other peers; the more peers the better but complex management & harder to secure

$\text{Timeout}_{\text{Int}} = 4 \cdot \text{devRTT} + \text{estRTT}_{\text{prev}}$

$\text{estRTT} = (1 - \alpha) \text{estRTT}_{\text{prev}} + \alpha \cdot \text{sampleRTT}$

$\text{devRTT} = (1 - \beta) \text{dev}_{\text{prev}} + \beta \cdot |\text{sampleRTT} - \text{estRTT}|$

RTT: time for client to travel from client to server and back

NP needs 2 RTTs per object, P needs 3 RTTs total

to get IP address for amazon.com:

1. client queries root server for DNS server

2. client queries TLD server for auth name $\rightarrow$ TLD = .com, .edu, etc

3. queries auth name server to get IP address $\rightarrow$ auth name server is basically host's internal DNS server

multiplexing: (at sender) add source + dest IP addr, source + dest port #

demultiplexing: (at receiver) use header info to deliver received

segments to sockets

socket: acts like a door, essentially the interface between app and transport layer in a host

ACK - receiver sends acknowledgement (ACK) after successfully receiving packet; if sender gets no ACK by timeout, resends packet; more reliable but slower

NAK - receiver sends a no-ack when it detects missing or corrupted packet; efficient when some packet loss is acceptable; faster; more complex error detection is required

go-back-N: sender can have up to N unacked packets in pipeline; receiver only sends cumulative ACKs; sender has timer for oldest unacked packet; resends all unacked packets when timer expires

selective repeat: sender can have up to N unacked packets in pipeline; receiver sends individual ACK for each packet; sender maintains timer for each unacked packet and resends individual packet when its timer expires

flow control	congestion control
purpose: prevents sender from overwhelming receiver	prevents excess traffic from overwhelming network
cause: receiver's queue limit	network having too much data
control: receiver shares cwnd (receiver window size)	sender adjusts cwnd (content window size)
effect: efficient reliable comm	reduce packet loss + delays
how: manages data flow between sender/recvr	prevents excess traffic by throttling sender

TCP Tahoe: loss indicated by timeout or triple duplicate ACKs drop cwnd to one mss and enter slow start phase and reset ssthresh to half of prev cwnd $\rightarrow$ slow start = exponential growth

TCP Reno: 3 dup ACKs does ssthresh = cwnd/2 and cwnd = ssthresh + 3 mss (resets when timeout) and cwnd grows linearly; congestion avoidance is linear growth $\rightarrow$ Reno enters fast recovery

Reno $>$ Tahoe because faster and more efficient

Tahoe timeout or triple duplicate ACKs:

ssthresh = 1/2 cwnd; cwnd = 1 mss, then slow start

TCP Reno timeout = same as Tahoe; 3 ACKs: ssthresh = 1/2 cwnd, cwnd = ssthresh + 3

we have 35 users who are active 10% of the time, what is prob 10 of them are active: $P(x=10) = \binom{35}{10} \cdot 0.1^{10} \cdot (1-0.1)^{25}$

end-to-end congestion control: no explicit feedback from network; congestion inferred from end-system loss and delay; approach taken from TCP

network-assisted congestion control: routers provide feedback to end systems; single bit indicating congestion; explicit rate for sender to send at

network layer functions:

forwarding: (data plane) move packets from router's input link to appropriate router output link

routing: (control plane) determine route taken by packets from source to destination

data plane: local per-router function that determines how datagram arriving on router input port is forwarded to router output port

control plane: network-wide logic that determines how datagram is routed among routers along end-to-end paths from source host to dest host

two types of routing protocol: **global** - all routers have complete link cost info; use Dijkstra (link state) algorithm

distributed - routers only know neighbor's cost; use distance vector algorithm (count to ∞)

$\rightarrow$ **count to infinity problem**: bad news travels slow (when a link goes down) routers will slowly increment the cost to a destination that is unreachable without realizing it is unreachable; causes slow convergence and incorrect routing tables for a long period of time

two types of network service architectures that use control plane:
datagram network (internet): no call setup at network layer; routers have no state about end-to-end connections; packets are forwarded using IP destination address (carried in header)
virtual circuit network (connection-oriented): call setup and teardown before data can flow; packets carry VC identifier not dest host addr

no strict time requirement, elastic service, many link types
no uniform service; "smart" end systems (computers); simple in network, complex at edge
strict timing requirements; need for guaranteed service; "dumb" end systems (telephones); complex in network

subnet: device interface that can physically reach each other without passing through an intervening router
example: given 69.168.0.34/29, we can figure out # of hosts supported, network ID, allowable IP addresses
address = 01000101.10101000.00000000.00100010
netmask = 11111111.11111111.11111111.11110000
net addr = 01000101.10101000.00000000.00100000 = m, 32
net addr bits = 29, host bits = 3 so $2^3 - 2 = 6$ hosts
000 and 111 are reserved (addr and broadcast)

Dynamic Host Config Protocol (DHCP): server that uses UDP to assign IP addresses to devices on a network and gives DNS addr
→ connecting laptop will use DHCP to get addr of first-hop router, IP, and DNS server; DHCP server responds with client's IP addr, IP address of first-hop router for client, & name & addr of DNS server

ICANN: allocates IP addresses and manages DNS root server, basically they give network subnet part of IP address
"Internet Corporation for Assigned Names and Numbers"

network address translation (NAT): allows multiple devices on a private network to access the internet using one public IP address
→ for outgoing datagrams, replace source IP addr + port # with NAT IP addr + new port #
→ for incoming datagrams, replace IP addr + port # as stored in NAT table and send to client
pros: saves public IP addr space, adds extra layer of privacy, delays urgency of upgrading IPv4 to IPv6
cons: violates end-to-end argument, complicates network configuration, reduces performance slightly

IPv6 upgrades: address size goes from 32 to 128 bits; hexadecimal instead of decimal notation (better scalability); simplified fixed-length header; fragmentation done by sender (not also routers); no NAT required so end-to-end connectivity

fragmentation: process of breaking large datagram into smaller pieces; computationally expensive (not required in IPv6); tracked using a fragment offset for order and More Fragment (MF) bit set to one if more fragments to follow

per-router control plane: each router is autonomous and runs its own forwarding table + route calculation

software-defined networking (SDN) control plane: one logically centralized controller making routing decisions; routers/switches become "dumb" forwarders
openflow - an open standard way to do SDN; stores flow rules: contains a match field (like MAC/IP addr), priority, actions, counters, & timeouts

hierarchical routing: aggregate routers into autonomous systems (ASs)
→ each AS has a group of routers under same admin control
→ routers in same AS use same intra-AS routing protocol
→ gateway routers are on edges of AS and linked to other gateways; perform inter-AS routing with other gateways
→ intra prioritizes performance, inter prioritized policy

priority rank when router selects a path:
1. local preference value attribute (policy decision)
2. shortest AS-path 3. closest next-hop rtr (hot potato)

intra: OSPF, RIP, EIGRP inter: BGP (AS-path stores ASs for loop)

link layer: responsible for transferring from one node to physically adjacent node over a link; implemented in a chip; high error rate on wireless links

point-to-point link: connects exactly two devices, no addresses needed beyond endpoints; low overhead; example: a direct fiber cable between 2 routers

broadcast link: connects multiple devices on network; everyone needs MAC address and find each other with protocol

types of error checking in link layer:
parity: detect and correct single bit errors with 2D bit parity
cycle redundancy: better error detection, can be put in hardware (good), and no need for connection
internet checksum: detect errors in transmitted part, implemented in software
forward error: detect and correct errors but more bandwidth overhead and computation complexity

scenario: client requests www.google.com webpage
1. client attaches to a network and broadcasts a DHCP req encapsulated in UDP (trans layer), IP (net layer), and ethernet frame (link layer); DHCP server receives + demuxes req, and assigns client: IP addr for client, IP addr of first hop router, & IP addr of DNS server
2. client broadcasts ARP query for MAC addr of first-hop router
3. client creates DNS query in UDP, IP (dest=DNS) & ethernet; and DNS server replies with google.com's IP address
4. client establishes a TCP connection with google web server; client sends TCP SYN, server responds with TCP SYNACK, client acks with TCP ACK
5. client sends HTTP request through socket, server sends back HTML

three broad types of MAC protocols:
channel partitioning: divide channel into smaller pieces and allocate pieces to nodes; inefficient, inflexible when adding/removing users, not scalable

taking turns: a control token passed around sequen

random access: collisions allowed and "recovery"

carrier sense mult access (CSMA): nodes listen before transmitting and defer when channel is busy, used in wired LANs efficiency = $1/(1+5t_{prop}+t_{trans})$

glotted aloha: nodes transmit into slots and retrans if collision detected; decentralized and simple

$P(\text{given node has success}) = p(1-p)^N$ where $N = \# \text{ nodes}$
 $P(\text{any node has success}) = Np(1-p)^{N-1}$

ARP table - one for every IP node, used to map MAC/IP address

switches vs routers - both are store and forward
→ routers: network layer → switches: link layer
o routers: compute fwding tables using routing algos + IP address
o switches: learn fwding tables using flooding, learning, & MAC address

Internet Control Message Protocol (ICMP): used by hosts + rtrs to comm network level info; ICMP info carried in IP datagrams

Management Information Base (MIB): like a dict of the structure and types of data that network devices can be configured with

MACs are portable, IP addresses are attached to local IP subnet